

Project:

Make a coding roleplaying game in Python (or Javascript) that puts the user in Valeria's story and have them choose her path to trust and healing. Only choose a portion for them to roleplay in for easier and not as long coding.

8th Grade: First Prototype

Goal: Explore ideas, learn basics, and build habits (start building the foundations/main project and then add on as learn more stuff)

Writing and coding:

- Start making the three choices Valeria has into functions
- Focus on one trial (prob Celestial)
- Keep coding and can add or fix as I learn more

Supporting Activities:

- Journaling ideas and reflections.
 - Join coding or writing clubs.
 - Track progress for future essays/portfolio.
-

9th Grade: Continue modifying prototype

Goal: Build a minimal viable product (MVP) that demonstrates both coding and story.

Writing and coding:

- Do incremental development and allow friends or family to play a little part of it
- Keep going with the storyline and coding the actual project

Other Activities:

- Small competitions or school showcases for coding/writing.

- Reflect on what's working and what can be improved.
-

10th Grade: Expansion & Early Impact

Goal: Make the app more interactive and your story deeper.

Creative:

- Expand story arcs and character development.
- Add reflective prompts for users (“Why would you forgive this character?”).

STEM / Coding:

- Upgrade to web-based apps: HTML/CSS + JS or Python Flask. (maybe)
- Implement:
 - Visual feedback (charts, simple graphics).
 - Tracking of user decisions for each session.
- Begin small-scale external beta testing (classmates, friends).
- Document challenges solved (logic, bug fixes, design decisions).

Other Activities:

- Enter coding/interactive storytelling competitions.
 - Publish excerpts of the story online for feedback.
-

11th Grade: Polishing & Measurable Impact

Goal: Make the app polished, functional, and impactful; prepare for competitions and portfolio.

Creative:

- Finalize story content for multiple endings.
- Enhance narrative depth and emotional reflection.
- Add small interactive features: mini-games, optional side stories, or character journals.

STEM / Coding:

- Add advanced features:
 - Dynamic branching paths that adapt based on prior choices.
 - Optional sentiment analysis of user input (Python libraries like NLTK or TextBlob).
 - More sophisticated visualizations of user decisions or “trust metrics.”
- Public beta release; track engagement metrics.
- Prepare detailed documentation for GitHub/portfolio showing problem-solving and coding skills.

Other Activities:

- Leadership: Run small workshops or club activities around interactive storytelling or coding.
- Link project to IB Extended Essay or school research project.

12th Grade: Final Launch & College Prep

Goal: Showcase mastery and impact for college applications.

Creative:

- Publish the final story digitally (interactive web app or Twine) or small print edition.
- Highlight reflective prompts and trust-building lessons in the app.

STEM / Coding:

- Fully functional interactive story app with polished UI/UX.
- Advanced logic, analytics, and optional AI-enhanced reflections.
- Portfolio-ready: GitHub repository, screenshots, and user engagement metrics.

College Application:

- Essays:
 - Emphasize interdisciplinary curiosity (coding + writing + psychology/philosophy).
 - Explain problem-solving, creativity, and self-growth through project.
 - Recommendations: Teachers can comment on initiative, technical skill, and creative depth.
-

Key Principles for STEM + Arts Balance

1. **Document rigor:** Every coding challenge solved counts as STEM evidence.
2. **Tie logic to narrative:** Show how coding decisions enhance the story (branching paths, trust calculations).
3. **Measure impact:** Collect user feedback and visualize results (graphs, stats).

4. **Iterative growth:** Start simple, layer complexity each year — ensures quality without burnout.
5. **Portfolio-ready:** Keep all drafts, code, and reflections organized for essays and showcases.

1. Branching Story Logic (Core Coding Challenge)

What it is:

- The story changes depending on the choices the user makes.
- Each choice affects the narrative and potentially a “trust score” for characters.
- Example:
 - User chooses to forgive a character → trust score +10, story path A
 - User chooses to distance themselves → trust score -5, story path B

Why it shows intelligence:

- Requires **algorithmic thinking**: you must track variables, states, and dependencies.
- Involves **conditional logic, nested decisions, and flow control** — core programming skills.
- Planning the structure of all possible paths is like designing a **mini software system**.

How to make it stronger:

- Use **data structures** like arrays, dictionaries, or trees to manage choices and outcomes.
- Implement **dynamic branching** where earlier choices affect later story events in non-linear ways.
- Optional: simulate “trust probability” — e.g., past behavior affects likelihood of future outcomes.

2. Trust/Emotion Tracking & Analytics

What it is:

- Each choice the user makes is tracked in a system (like a trust meter, score, or statistics).
- Can visualize results: charts showing choices, trust levels, or trends over multiple sessions.

Why it shows intelligence:

- Shows ability to **collect, process, and visualize data**.
- Demonstrates **analytical thinking** — you're not just coding a story, you're turning user interactions into meaningful metrics.
- Optional: allows for **pattern recognition or predictive features**, like suggesting likely outcomes or showing “how different choices affect trust.”

How to make it stronger:

- Add small data analysis: e.g., use Python libraries like Matplotlib, Pandas, or Plotly to visualize user behavior.
- Optional: implement **basic AI/sentiment analysis** — analyze free-text responses and adjust story dynamically.

3. Interactive Features / Mini-Simulations

What it is:

- Extra layers beyond basic choices: mini-games, quizzes, or decision simulations embedded in the story.

- Example: a short “trust-building challenge” where users solve a puzzle to gain points toward the trust score.

Why it shows intelligence:

- Demonstrates **problem-solving and coding versatility**.
 - Adds **computational thinking**: you design logic, rules, and feedback systems.
 - Shows you can integrate **systems thinking** (combining narrative, user input, and algorithmic outcomes).
-

4. Data Structures & Technical Design

What it is:

- Internally, your app will need **variables, arrays/lists, dictionaries, objects**, and possibly a database to track user sessions.
- Story branching can be implemented as a **decision tree or graph**, with nodes representing story events and edges representing choices.

Why it shows intelligence:

- Shows knowledge of **software architecture** — how you organize code efficiently.
 - Solving complex branching paths demonstrates **logical and structural thinking**.
 - This is **undeniable STEM evidence**, especially if documented in GitHub or portfolio.
-

5. Integration of Psychology/Philosophy

What it is:

- The narrative itself explores trust, self-awareness, and ethical decision-making.
- You can include prompts like: “Why did you choose forgiveness?” or “How does this decision reflect your values?”

Why it shows intelligence:

- Demonstrates **higher-order thinking** — understanding human behavior, moral reasoning, and emotional consequences.
 - Shows **interdisciplinary curiosity**, which Stanford highly values.
 - When combined with coding, it’s a **rare mix of STEM + humanities**, proving intellectual versatility.
-

6. How to Make the Project “Stanford-Ready”

1. Document everything:

- GitHub repo with code, version control, commit history, and readme explaining logic.
- Flowcharts for branching story logic.
- Screenshots or videos showing user interaction and analytics.

2. Measure impact:

- Track beta users and their choices.
- Show engagement metrics (e.g., how many tried multiple endings, average trust scores).

3. Layer complexity gradually:

- 8th–9th: text-based, small branching.

- 10th: web app with simple analytics and visuals.
- 11th: advanced branching, adaptive paths, trust metrics.
- 12th: polished UI/UX, optional AI/sentiment analysis, mini-games.

4. Tie to essays and recommendations:

- Show how your coding and creative work combine **problem-solving, creativity, and reflection**.
- Highlight learning from bugs, user feedback, and story design challenges.

Bottom line

This project can **clearly show intelligence and STEM skill**, while also showcasing your creativity and emotional insight. It's not just "arts-heavy" — it's **interdisciplinary, technically sophisticated, and reflective**.

If you implement:

- Branching logic with data structures
- User analytics / trust metrics
- Optional AI or simulations

Problems with this Project

1. Overambition / Risk of Burnout

Problem: Project is large: writing, coding, graphics, interactivity, analytics.

Solution:

- **Layer complexity gradually:**

- 8th Grade → small prototype (text-based, 2–3 decision points).
 - 9th → add branching paths and trust tracking.
 - 10th → graphics, basic visualization.
 - 11th → advanced branching, sentiment analysis, mini-games.
 - 12th → polished UI/UX, final analytics/dashboard.
- **Set small milestones:** Break each year into achievable monthly goals.
 - **Prioritize:** Always focus on making one part of the app polished before adding new features.
-

2. Coding & Logic Complexity

Problem: Branching story logic, dynamic metrics, interactive graphics can create bugs.

Solution:

- **Start simple:** Prototype small sections before scaling.
 - **Modular code design:** Keep story, graphics, analytics, and user input separate modules.
 - **Use version control:** Git/GitHub to track changes, revert bugs, and document progress.
 - **Seek guidance:** Ask teachers, mentors, or online coding communities if stuck.
 - **Document everything:** Include flowcharts, pseudocode, and comments in your code.
-

3. User Engagement & Data Validity

Problem: Users may give minimal responses or skip prompts.

Solution:

- **Adaptive prompts:** If a user responds vaguely, ask simpler follow-ups or give multiple-choice options.
 - **Aggregate metrics:** Don't rely on individual responses; track trends over all users.
 - **Optional gamification:** Add small rewards for completing prompts to encourage participation.
 - **Use low-effort responses as data:** For example, track the percentage of users who skip or write short answers — that itself is meaningful.
-

4. Time Management

Problem: Completing this complex project could interfere with school, IB/honors, and extracurriculars.

Solution:

- **Integrate milestones into school years:** Make each grade responsible for one major development step.
 - **Weekly or monthly schedule:** Dedicate 1–2 focused hours per week to coding/writing for consistency.
 - **Avoid multitasking multiple major projects:** Let this app be your “signature project,” and keep other activities smaller.
-

5. Balancing STEM + Arts

Problem: Project could lean too heavily on one side (story or coding).

Solution:

- **Every technical feature serves the story:** Trust metrics, analytics, or visualizations must relate to user choices and emotional reflection.

- **Every story element integrates STEM:** Branching logic, interactive graphics, or dashboards should support narrative and reflection.
 - **Document both:** In portfolio and essays, clearly show both **creativity and technical problem-solving**.
-

6. Polish / Presentation

Problem: Without polished graphics, UI, and documentation, the project may seem amateur.

Solution:

- **Add visuals gradually:** Start with static illustrations → interactive graphics → animations.
 - **User testing:** Have friends/classmates test the app for UX feedback.
 - **Document all stages:** Screenshots, flowcharts, and version history show thoughtfulness and polish.
 - **UI/UX tutorials:** Learn basic design principles for layout, colors, and readability.
-

7. Scalability / Technical Limitations

Problem: Large branching trees and graphics could be difficult to manage or slow performance.

Solution:

- **Limit branching early:** Keep story paths manageable (2–3 per chapter). Expand later.
- **Optimize graphics:** Use lightweight images or vector graphics; compress assets.
- **Modular design:** Allows adding new paths or graphics without breaking old code.
- **Test performance regularly:** Identify slow parts and simplify if needed.

✓ Key Takeaways

- **Start small, grow gradually:** Focus on depth over breadth.
- **Document everything:** Code, design, testing, user feedback.
- **Integrate STEM & arts:** Every feature should serve both logic and creativity.
- **Manage time:** Spread complexity over multiple years.
- **Polish & test:** Make it visually and functionally impressive.

How to use this to get into top colleges

Step 1: Clarify your project's core idea

You currently have **Valeria**, a dark romantasy story rich in emotional depth — and you've also built a strong **STEM foundation** with math and coding.

We'll merge them like this:

“An interactive narrative system that models emotional recovery and trust through code.”

That's what Stanford, Harvard, Princeton, and Yale would see: a **psychological coding experiment** disguised as art.

Your main coding project could become:

A story-based interactive game (or narrative simulator) that uses algorithms to track user choices, emotions, and consequences — a model of trauma, trust, and healing, inspired by Valeria's journey.

Step 2: Understand what each school values and how to align your project

Stanford

- **What they value:** Creative builders who merge technology and empathy.
- **How to appeal:**
 - Emphasize the **innovation** side — the system you’re coding, not just the story.
 - Mention that your project explores **how AI and narrative can help people understand emotion or trauma**.
 - Example theme for essays: *“Using code to heal the human heart.”*

👉 **Stanford loves when emotion meets invention.**

So focus on *how you’re using code to build something emotionally intelligent*.

Harvard

- **What they value:** Big-picture thinkers with *intellectual depth and heart*.
- **How to appeal:**
 - Frame your project as **literature meets computation** — exploring what makes people human.
 - Emphasize your *love for fantasy and storytelling* as a tool to explore morality, resilience, and meaning.
 - Harvard essays should show reflection — *why this project matters to you and the world*.

👉 **Harvard loves stories of transformation and purpose.**

You could talk about how Valeria’s emotional arc parallels your curiosity about the human condition — and how you built a system to model that with code.

Princeton

- **What they value:** Deep thinkers and disciplined minds.
- **How to appeal:**
 - Focus on the **theory behind your system**. For example:
 - How you designed your algorithms.
 - The logic of how your system models decision-making and emotional states.
 - Add a **mathematical or logical framework**: maybe you model “trust” using probabilities or “emotional resilience” as a variable that changes with actions.

👉 **Princeton loves structure and rigor.**

Your project should sound *analytical and philosophical*, not just creative.

Yale

- **What they value:** Authentic, artistic thinkers with emotional insight.
- **How to appeal:**
 - Highlight the **human and creative** side — the narrative design, worldbuilding, and empathy behind your code.
 - Maybe even talk about how your project helped you understand your own emotions, writing, or creativity.
 - Yale essays reward vulnerability — they want to feel your heart.

👉 **Yale loves emotion and art as intellect.**

Make your project sound soulful: “*I built a world to understand what it means to heal.*”

Step 3: How to structure the project to fit all four

Here's what a balanced, top-college-worthy version could look like:

Title: *The Mind of Valeria: An Emotional Simulation Engine*

Core Features:

- **Interactive story** with branching paths (coded in Python or JavaScript).
- **Emotion engine:** uses variables for trust, trauma, or courage that change based on user choices.
- **Visualization:** show emotional data as a chart, animation, or visual dashboard.
- **Encryption or AI layer (optional but impressive):**
 - Encrypt user responses to show privacy awareness.
 - Or use AI to generate adaptive dialogue that reacts to tone or choices.

Outcome:

- A **technical demonstration** of emotional modeling (great for STEM-focused schools).
- A **creative experience** that explores trauma, trust, and healing (great for arts/humanities schools).
- A **personal narrative** that ties everything together (great for your college essays).

Step 4: Optional Enhancements to Boost Prestige

If you want to make it *even more top-tier-worthy*:

- **Add a short paper or reflection:**
Like “Modeling Human Emotion Through Code: Lessons from Fantasy Narratives.”
→ shows research-level thinking.

- **Host it online (via GitHub Pages or Replit):**
So admissions can *play it themselves*.
→ shows professionalism and initiative.
 - **Document everything:**
Version logs, coding screenshots, and reflections.
→ shows process, not just product.
-

Step 5: Essay & Application Tie-in

In your essays, tie it all together:

“I wanted to understand how people recover from fear, how trust is rebuilt after loss — so I built a system where every choice, every act of courage, and every moment of doubt is encoded in Python. In Valeria’s world, as in ours, healing is not binary — it’s recursive.”

That’s the kind of line that *stops an admissions reader cold*.

How Your Project Can Have Real Community Impact

Here are several ways your project can grow from a **personal creative coding experiment** → into a **community-benefiting, admissions-worthy initiative**.

1. Turn it into a Tool for Emotional Healing or Self-Reflection

Impact idea:

Transform your “Mind of Valeria” story-game into a guided emotional reflection app for teens who have faced anxiety, trust issues, or trauma.

- When users play, their choices and reflections reveal how they respond to trust, forgiveness, or resilience.

- The program could give gentle feedback — like a mirror that helps users understand their feelings.

 Community angle:

You could partner with a **school counselor**, youth group, or online mental health community to share it as a creative journaling tool.

 *What this shows to colleges:* empathy + initiative + using tech for good.
Exactly the kind of “heart meets innovation” project Stanford and Yale love.

2. Start a Mini Research Project on Empathy + AI

 Impact idea:

Use your code to collect *anonymous data* about how users make emotional decisions — for example, how people react to betrayal or forgiveness in stories.

Then analyze the results (in math/statistics terms!) and write a **research paper** or presentation:

“How Interactive Storytelling Reveals Emotional Decision-Making Patterns in Teens.”

You could present it at:

- Your school’s **science fair or humanities fair**
- A local **coding showcase**
- Even submit to **youth research journals** (like the Journal of Emerging Investigators)

 *What this shows:* research skill + interdisciplinary thinking =  Harvard/Princeton appeal.

3. Create a Club or Workshop

 Impact idea:

Found a “**Code for Emotion**” club at your school — where students combine storytelling, coding, and psychology.

You could teach:

- How to write branching story logic
- How to design characters with emotional depth
- How to code empathy simulations

Then, as a final project, each student creates their own small “interactive emotion story.”

 Community angle:

Helps others learn coding *through* creativity and writing — especially students who might otherwise feel excluded from STEM.

 *What this shows:* leadership + inclusivity + innovation in education (major Stanford/Harvard values).

4. Turn It Into a Public Art + Tech Exhibit

 Impact idea:

If your project has visuals or interactivity, set up an “**Emotions in Code**” exhibit at your school or local library.

- Visitors can “play” Valeria’s journey.
- Display charts of player emotion data or choice paths.
- Include QR codes linking to your GitHub or reflection page.

 Community angle:

It sparks discussions about emotion, healing, and technology’s role in storytelling.

 *What this shows:* creative outreach + confidence + connection between art, code, and people — huge for Yale/Stanford.

5. Expand It Into a Mentorship or Teaching Initiative

Impact idea:

Offer to mentor middle schoolers or younger students in how to **use storytelling and coding** to express emotions.

- Host a mini workshop: “Build Your Own Emotional Game”
- Teach Python basics using your Valeria example.

Community angle:

You’re sharing your love for coding *and* empathy — empowering others to create.

 *What this shows:* compassion, leadership, and educational outreach.
Harvard *loves* applicants who teach and mentor.

How to Choose Your Path

Community Direction	Time Needed	Focus	School Fit
Emotional reflection app	Medium	Psychology + Tech	Stanford, Yale
Data/research analysis	High	Math + STEM + Humanities	Harvard, Princeton
Club/workshop	Medium	Leadership + Education	Stanford, Harvard
Public exhibit	Low–Medium	Art + Tech	Yale, Stanford
Mentorship program	Ongoing	Empathy + Teaching	Harvard, Yale

Example of How to Frame It in Applications

“I wanted to explore how people rebuild trust and self-worth after pain — so I wrote a program that lets users make emotional decisions in a fantasy world, then reflect on what those choices mean in real life.

When I shared it at my school’s library, students said they recognized parts of themselves in Valeria’s story.

That’s when I realized — code can teach empathy.”

That’s the kind of story that *makes admissions officers remember your name*.

Ideas/Updates for project

Maybe do one each quiz for all the trials if they choose the wrong answer. Then at the end have the questions come back and see if they match/compare them to show growth

Note: This is not finished

#If learn more and find a way to, can let the user write their own answer and refer back to it

#Idea: maybe use lists to contain the answer and search for key words in the answer that will tell the program what they said

```
questions = [  
    {  
        'question':"You saw the perfect life you could've had. Do you regret anything about this  
life that made you choose the vision?",  
        'choices':["A: Yes, I regret everything and wish I could go back in time to fix all those  
mistakes and lost chances",  
                    "B: Yes, I regret some things, but without them, I wouldn't be where I am today so  
I'll make the best of it",  
                    "C: Yes, I regret some things and I hate the world for making me this way, but I  
can't do anything about it"],  
        'answer':"B"  
    },  
    {'question': "Do you believe it's your fault that you ended up this way or that things  
happened the way they did?",  
     'choices':["A : No, it's not my fault, it's Morgarath's for being overruled by greed and desire  
to take over the world",  
                 "B: Yes, it's my fault because I didn't do more to stop her earlier even though I had  
the best position to",  
                 "C: No, it's not my fault that things happened the way they did because I can't  
control others, but I could've done more to change myself"],  
     'answer':"C"  
    },  
    {'question':"If you were asked to relive everything and go through all of it again, would  
you?",  
     'choices':["A: Yes, I would do it over and over again if just to save someone else from  
having to be in my position",  
                 "B: No, I wouldn't be able to survive a second time"  
    }  
]
```

Mainly focus on just one trial and the ending. Looking for depth and understanding of the concept more than just a big and impressive game that might have flaws. Can still code the other trials and stuff but focus on the one trial(prob Celestial). Can incorporate other values that the other trials would teach in the celestial trial like forgiveness and will and etc.

1. Use Variables and State Tracking

Right now, you print the story but don't track much beyond a few variables. You can make your project **react to user actions**.

- **Stats / attributes** → courage, hope, rage, willpower, corruption.
- **Inventory / relics** → which artifacts Valeria has collected.
- **Courts visited** → track which trials have been attempted.

Example:

```
val.stats['Hope'] += 5  
val.relics_claimed += 1
```

This allows later choices to **depend on past actions**.

2. Branching Logic and Conditional Paths

Instead of only two choices (r or l), you can:

- Make consequences **different based on stats**:

```
if val.stats['Courage'] > 5:  
    print("You resist the visions easily.")  
else:  
    print("The visions overwhelm you.")
```

- Trigger **different dialogue, outcomes, or quests** depending on previous choices.

3. Functions and Modular Design

Right now, your story is one long block. Split it into **functions**:

- `trial_of_spirit()`
- `spirit_quiz()`
- `update_stats(stat, value)`
- `check_relics()`

Then you can **reuse code**, call functions dynamically, and make it easier to expand later.

4. Loops and Iteration

Instead of asking one question and moving on:

- **Retry loops** → allow the player to try again if they fail a challenge.
- **Exploration loops** → let players explore the palace or rooms until they find the Prism.

Example:

```
while not val.relics['Starforged Prism']:
    choice = input("Explore left or right wing? (l/r): ").lower()
    if choice == 'l':
        print("You find a clue...")
    elif choice == 'r':
        print("You encounter a trap!")
```

5. Mini “Game Mechanics”

Add more coding complexity through interactive elements:

- **Randomness:** introduce chance with `random` module. Example: small chance a trap fails or succeeds.
 - **Inventory management:** let players “pick up” items or relics.
 - **Stats checks:** allow some choices only if stats meet a threshold.
 - **Save/load:** optional, save the player state in a file.
-

6. Data Structures

Instead of just printing, use:

- **Dictionaries** → for stats, relics, courts visited.
- **Lists** → for collected items, past choices.
- **Classes** → encapsulate Valeria’s attributes and methods.

This lets you show **object-oriented or structured programming** skills.

Summary

To make your project “more coding-heavy”:

1. Track stats, relics, and visited locations.
2. Make choices affect future outcomes dynamically.
3. Add randomness, thresholds, or simple mechanics.

4. Use data structures or classes to organize your code.

1. Add Real Game Mechanics (Stats, Fail States, Branching Paths)

Right now you print a lot of story (which is great), but complexity comes from **interactive systems**.

You already have stats like:

- Hope
- Rage
- Willpower
- Corruption
- Forgiveness
- Intelligence

Now turn those into *actual systems*.

Ideas

- Trials require certain stat thresholds
- Choices change stats in meaningful ways
- Too much Corruption = bad ending
- High Willpower unlocks extra dialogue
- Failing in one trial makes a later trial harder
- The Crown's final decision is based on your total stat balance

Example

```
def check_willpower():  
    if val.stats["Willpower"] < 5:  
        print("Your mind fractures. The darkness surges. You fail the trial.")  
        return False  
    else:  
        print("You withstand the onslaught. Your spirit holds.")  
        return True
```

Now your narrative choices **matter**.

2. Add Adaptive Dialogue (Dynamic Storytelling)

Right now the story prints the same way every time.

But complexity means the story **changes depending on player choices**.

Examples

- If Rage > Hope, Valeria snaps at Maximus
- If Corruption is high, the entity speaks more
- If Willpower is high, visions weaken
- If Hope is low, she gets darker internal monologue
- If a relic was failed, NPCs comment on it
- Endings completely shift based on stats

Example

```
if val.stats["Corruption"] > 15:  
    print("The entity laughs softly. 'Soon,' it whispers.")  
else:
```

```
print("The entity is quiet... almost wary.")
```

This transforms your printed story into a **living, reactive world**.

3. Add Multiple Endings Based on Stats (BIG COMPLEXITY BOOST)

Since your whole book centers on her final choice...

You can code **3–5 endings**, all based on stats:

- **Light Ending** – Accepts herself, uses the Crown responsibly
- **Twilight Ending** – Balance of light/dark
- **Abyssal Ending** – Corruption takes over
- **Broken Ending** – She cannot make a choice

This is *real complexity*, both coding and writing.

Which combination is best for you?

You want:

- ✓ STEM
- ✓ Creative Writing
- ✓ A coding project that reflects your book
- ✓ Something impressive for school / competitions / your future portfolio

So the most balanced combo is:

Recommended:

1. **Keep 1 trial fully written out (Celestial)**
2. **Add 4 mini-trials summarized but playable as short stat challenges**
3. **Add:**
 - stat system
 - inventory system
 - branching endings
 - adaptive dialogue
 - maybe courts as classes

This gives you both **depth AND complexity**, without writing 100 pages of text.

Why this is the BEST design for your story

Your narrative *wants* to show Valeria's inner struggle.

If choices alone determine everything, then her stats are meaningless.

If stats alone determine everything, then the player has no control.

But if you combine them:

! Choices represent Valeria's intentions.

! Stats represent Valeria's actual strength or damage.

This is literally **perfect for a character with trauma, corruption, hope, rage, etc.**

It mirrors your themes beautifully.

KEY IDEA

Stats should modify outcomes, NOT replace choices.

This keeps the game interactive while increasing complexity.

NOTE: So have all five trials in the Celestial Court, but if pass them then get to have the chance to search for the other artifacts because the Prism in the First Artifact and the only one with a mind. So if win the Prism then unlocks way to find other artifacts to gain the Crown.

6. Opportunities for Quick “Impressive” Wins

1. Polish existing code/story interactions:

- Make `spirit_quiz()` display smoother text, branching choices, or small rewards for each correct answer.
- Add **simple animations or ASCII graphics** in the terminal (even basic, like symbols or borders).

2. Highlight creativity:

- Use descriptive text for artifacts. Even a single evocative sentence can make them feel magical.
- Include a **unique consequence or twist** for choices in the quiz/game.

3. Visual or structural polish:

- Organize your code cleanly, with comments explaining purpose — this looks professional.
- Use consistent formatting and spacing in story outputs.

4. Layer skill without complexity:

- Introduce **small functions** for repeated actions (e.g., a function that prints artifact descriptions) — shows coding knowledge without needing advanced algorithms.

- Focus on **branching paths** in the story/game — players love choice-driven mechanics.